



Plan your trip with Kayak

Auteur : Aicha FATHALLAH - Formation Fullstack Data Engineering

Objectif : Recommander les 5 meilleures destinations françaises et les 20 meilleurs hôtels associés, en combinant données météo (API OpenWeatherMap) et données hôtelières (scraping Booking.com), stockées sur AWS S3 / RDS.

Pipeline : Géocodage -> API Météo -> Scraping Hôtels -> Nettoyage & Scoring -> S3 (Data Lake) -> RDS (Data Warehouse) -> Visualisation

```
In [ ]: # Mise à jour de pip puis installation des dépendances
%pip install --upgrade pip
%pip install -r requirements.txt
```

1. Installation & Imports

```
In [ ]: %pip install selenium webdriver-manager boto3 sqlalchemy psycpg2-binary plotly geopy python-dotenv -q
```

```
# Bibliothèques standard Python
import json, time, re, os
from io import StringIO
from pathlib import Path

# Manipulation de données
import pandas as pd      # DataFrames : Lecture, transformation, export CSV
import numpy as np       # Calculs numériques

# Requêtes HTTP
import requests          # Appels API REST (OpenWeatherMap)

# Visualisation interactive
import plotly.express as px # Cartes Mapbox interactives

# Géocodage
from geopy.geocoders import Nominatim # Conversion nom -> coordonnées GPS

# Scraping web
from bs4 import BeautifulSoup
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager

# Cloud AWS
import boto3
from sqlalchemy import create_engine

# Sécurité des clés API
from dotenv import load_dotenv
load_dotenv()
```

```
# Dossier de sortie des données (créé s'il n'existe pas)
DATA = Path("data")
DATA.mkdir(exist_ok=True)

print("Environnement prêt")
```

Note: you may need to restart the kernel to use updated packages.
Environnement prêt

2. Géocodage des 35 destinations (Nominatim)

Récupération des coordonnées GPS via l'API OpenStreetMap : pause d'1 seconde entre chaque ville pour respecter le rate limit.

La liste reprend exactement les 35 villes demandées.

```
In [ ]: # Liste officielle des 35 destinations attendues par Kayak
villes = [
    "Mont Saint Michel", "St Malo", "Bayeux", "Le Havre", "Rouen", "Paris",
    "Amiens", "Lille", "Strasbourg", "Chateau du Haut Koenigsbourg", "Colmar",
    "Eguisheim", "Besancon", "Dijon", "Annecy", "Grenoble", "Lyon",
    "Gorges du Verdon", "Bormes les Mimosas", "Cassis", "Marseille",
    "Aix en Provence", "Avignon", "Uzes", "Nimes", "Aigues Mortes",
    "Saintes Maries de la Mer", "Collioure", "Carcassonne", "Ariege",
    "Toulouse", "Montauban", "Biarritz", "Bayonne", "La Rochelle"
]

# Initialisation du géocodeur Nominatim (OpenStreetMap, gratuit, sans clé API)
# user_agent obligatoire pour identifier notre application auprès du serveur
geolocator = Nominatim(user_agent="kayak_project_agent")
villes_data = []

for ville in villes:
    try:
        # On precise "France" pour éviter les homonymes
        loc = geolocator.geocode(f"{ville}, France")
        if loc:
            villes_data.append({
                "city": ville,
                "latitude": loc.latitude,
                "longitude": loc.longitude
            })
            print(f"{ville}")
        else:
            print(f"{ville} - non trouvé")

        # Pause obligatoire : Nominatim limit a 1 requête/seconde
        time.sleep(1)
    except Exception as e:
        print(f"Erreur {ville}: {e}")

# Création du DataFrame et ajout d'un identifiant unique par ville
df_villes = pd.DataFrame(villes_data)
df_villes.insert(0, "city_id", range(1, len(df_villes) + 1))

# Sauvegarde locale pour ne pas avoir à relancer le géocodage
df_villes.to_csv(DATA / "villes_coords.csv", index=False)

print(f"\nGéocodage terminé : {len(df_villes)}/35 villes")
display(df_villes.head())
```

Mont Saint Michel
St Malo
Bayeux
Le Havre
Rouen
Paris
Amiens
Lille
Strasbourg
Chateau du Haut Koenigsbourg
Colmar
Eguisheim
Besancon
Dijon
Annecy
Grenoble
Lyon
Gorges du Verdon
Bormes les Mimosas
Cassis
Marseille
Aix en Provence
Avignon
Uzes
Nimes
Aigues Mortes
Saintes Maries de la Mer
Collioure
Carcassonne
Ariege
Toulouse
Montauban
Biarritz
Bayonne
La Rochelle

Geocodage terminé : 35/35 villes

	city_id	city	latitude	longitude
0	1	Mont Saint Michel	48.635954	-1.511460
1	2	St Malo	48.649518	-2.026041
2	3	Bayeux	49.276462	-0.702474
3	4	Le Havre	49.493898	0.107973
4	5	Rouen	49.440459	1.093966

3. Données météo : API OpenWeatherMap

Clé API chargée depuis config/openweather.env (variable OPENWEATHER_API_KEY).

Endpoint : data/2.5/forecast (prévisions 5 jours / 3 h, gratuit).

Les prévisions météo sont récupérées sur 5 jours via l'endpoint gratuit forecast d'OpenWeatherMap, l'API One Call sur 7 jours étant devenue payante (carte bancaire requise) ; ce choix couvre l'horizon disponible tout en maintenant un coût nul.

```
In [ ]: # Chargement de la clé API depuis Le fichier .env (jamais La vraie dans Le code)
load_dotenv(dotenv_path=Path("config/openweather.env"))
API_KEY = os.getenv("OPENWEATHER_API_KEY")
```

```

weather_results = []

for _, row in df_villes.iterrows():
    # units=metric -> températures en Celsius ; lang=fr -> descriptions en français
    url = (
        f"https://api.openweathermap.org/data/2.5/forecast"
        f"?lat={row['latitude']}&lon={row['longitude']}"
        f"&appid={API_KEY}&units=metric&lang=fr"
    )
    resp = requests.get(url)
    if resp.status_code == 200:
        previsions = resp.json()["list"]    # 40 créneaux : 5 jours / 3 h

        # Agrégation sur tout l'horizon disponible (correction : avant, un seul créneau)
        temps = [p["main"]["temp"] for p in previsions]
        pops = [p.get("pop", 0) for p in previsions]    # probabilité de pluie [0-1]
        pluie = [p.get("rain", {}).get("3h", 0) for p in previsions]    # volume de pluie (mm)

        weather_results.append({
            "city_id": row["city_id"],
            "city": row["city"],
            "temp": round(float(np.mean(temps)), 1),    # température moyenne sur l'horizon
            "rain_prob": round(float(np.mean(pops)), 2),    # probabilité moyenne de pluie
            "rain_volume": round(float(np.sum(pluie)), 1),    # volume de pluie cumulé (mm)
            "weather_description": previsions[0]["weather"][0]["description"]
        })
        print(f"{row['city']}")
    else:
        print(f"{row['city']} - code {resp.status_code}")

    time.sleep(0.2)    # rester dans Les Limites du plan gratuit (60 req/min)

# Fusion des données météo avec Les coordonnées GPS
weather_df = pd.DataFrame(weather_results)
if weather_df.empty:
    if not API_KEY:
        raise ValueError(
            "OPENWEATHER_API_KEY introuvable. Vérifiez que 'config/openweather.env' existe "
            "et contient la variable OPENWEATHER_API_KEY, ou exportez-la dans l'environnement."
        )
    else:
        raise ValueError(
            "Aucune donnée météo n'a été récupérée. Vérifiez que la clé OPENWEATHER_API_KEY est valide "
            "et que les requêtes vers OpenWeatherMap renvoient 200 (resp.status_code)."
        )

expected_cols = ["city_id", "temp", "rain_prob", "rain_volume", "weather_description"]
missing = [c for c in expected_cols if c not in weather_df.columns]
if missing:
    raise ValueError(f"Colonnes manquantes dans weather_results : {missing}")

df_weather_final = pd.merge(
    df_villes,
    weather_df[expected_cols],
    on="city_id",
    how="left"
)

df_weather_final.to_csv(DATA / "villes_meteo_coords.csv", index=False)

print(f"\nMétéo agrégée pour {len(df_weather_final)} villes")
display(df_weather_final.head())

```

Mont Saint Michel
St Malo
Bayeux
Le Havre
Rouen
Paris
Amiens
Lille
Strasbourg
Chateau du Haut Koenigsbourg
Colmar
Eguisheim
Besancon
Dijon
Annecy
Grenoble
Lyon
Gorges du Verdon
Bormes les Mimosas
Cassis
Marseille
Aix en Provence
Avignon
Uzes
Nimes
Aigues Mortes
Saintes Maries de la Mer
Collioure
Carcassonne
Ariege
Toulouse
Montauban
Biarritz
Bayonne
La Rochelle

Meteo agrégée pour 35 villes

	city_id	city	latitude	longitude	temp	rain_prob	rain_volume	weather_description
0	1	Mont Saint Michel	48.635954	-1.511460	17.3	0.0	0.0	partiellement nuageux
1	2	St Malo	48.649518	-2.026041	17.1	0.0	0.0	couvert
2	3	Bayeux	49.276462	-0.702474	18.0	0.0	0.0	partiellement nuageux
3	4	Le Havre	49.493898	0.107973	17.8	0.0	0.0	peu nuageux
4	5	Rouen	49.440459	1.093966	19.5	0.0	0.0	partiellement nuageux

4. Scraping Booking.com : Selenium (Chrome headless)

`requests` est bloqué par Booking (code 202 + JS dynamique) -> utilisation de Selenium. Stratégie : scroll progressif pour déclencher le lazy-loading, limité à 20 hôtels par ville, checkpoint CSV après chaque ville.

```
In [ ]: def scrape_hotels_selenium(city):  
        """  
        Scrape les hôtels d'une ville sur Booking.com via Selenium Chrome headless.  
  
        Pourquoi Selenium et pas requests ?  
        -> Booking.com charge son contenu via JavaScript (rendu dynamique).  
        -> requests ne récupère que le HTML statique : les cartes hôtels sont absentes.  
        Selenium pilote un vrai navigateur qui execute le JS.
```

```

Retourne : list[dict] {city, hotel_name, url, rating, description}
"""
opts = Options()
opts.add_argument("--headless")
opts.add_argument("--window-size=1920,1080")
opts.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36")

driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()), options=opts)
hotels = []
url = f"https://www.booking.com/searchresults.fr.html?ss={city.replace(' ', '%20')}}"

try:
    driver.get(url)
    time.sleep(5) # chargement initial

    # Scroll progressif pour déclencher le lazy-loading des cartes hôtels
    for i in range(4):
        driver.execute_script(f"window.scrollTo(0, {(i+1)*1000});")
        time.sleep(2)

    for card in driver.find_elements(By.CSS_SELECTOR, '[data-testid="property-card"]')[:20]:
        try:
            name = card.find_element(By.CSS_SELECTOR, '[data-testid="title"]').text.strip()
            hotel_url = card.find_element(By.CSS_SELECTOR, '[data-testid="title-link"]').get_attribute("href")

            # Note de l'hôtel (ex : "Superbe 9.4" -> 9.4)
            try:
                raw = card.find_element(By.CSS_SELECTOR, '[data-testid="review-score"]').text
                m = re.search(r"(\d[.]\d)", raw)
                rating = float(m.group(1).replace(".", "")) if m else 0.0
            except:
                rating = 0.0

            # Description / type de chambre (charge tardivement par JS, souvent N/A)
            try:
                desc = card.find_element(By.CSS_SELECTOR, '[data-testid="recommended-units"]').text.strip()
            except:
                desc = "N/A"

            if name:
                hotels.append({
                    "city": city,
                    "hotel_name": name,
                    "url": hotel_url,
                    "rating": rating,
                    "description": desc
                })
        except:
            continue
    finally:
        driver.quit()

    return hotels

```

```

In [ ]: OUTPUT_FILE = DATA / "hotels_booking_35_villes.csv"
        final_data = []

```

```

for _, row in df_villes.iterrows():
    print(f"[{row['city_id']}/35] {row['city']}...")
    try:
        results = scrape_hotels_selenium(row["city"])

```

```
# Ajout du city_id à chaque hôtel pour la clé de jointure
for h in results:
    h["city_id"] = row["city_id"]

final_data.extend(results)

# Checkpoint : sauvegarde après chaque ville
pd.DataFrame(final_data).to_csv(OUTPUT_FILE, index=False)
print(f" -> {len(results)} hôtels | total cumulé : {len(final_data)}")
except Exception as e:
    print(f"Erreur : {e}")

time.sleep(5)

df_hotels_raw = pd.DataFrame(final_data)
print(f"\nScraping terminé - {len(df_hotels_raw)} lignes")
display(df_hotels_raw.head())
```

[1/35] Mont Saint Michel...
-> 3 hôtels | total cumulé : 3
[2/35] St Malo...
-> 3 hôtels | total cumulé : 6
[3/35] Bayeux...
-> 3 hôtels | total cumulé : 9
[4/35] Le Havre...
-> 3 hôtels | total cumulé : 12
[5/35] Rouen...
-> 3 hôtels | total cumulé : 15
[6/35] Paris...
-> 3 hôtels | total cumulé : 18
[7/35] Amiens...
-> 3 hôtels | total cumulé : 21
[8/35] Lille...
-> 3 hôtels | total cumulé : 24
[9/35] Strasbourg...
-> 3 hôtels | total cumulé : 27
[10/35] Chateau du Haut Koenigsbourg...
-> 3 hôtels | total cumulé : 30
[11/35] Colmar...
-> 3 hôtels | total cumulé : 33
[12/35] Eguisheim...
-> 3 hôtels | total cumulé : 36
[13/35] Besancon...
-> 3 hôtels | total cumulé : 39
[14/35] Dijon...
-> 3 hôtels | total cumulé : 42
[15/35] Annecy...
-> 3 hôtels | total cumulé : 45
[16/35] Grenoble...
-> 3 hôtels | total cumulé : 48
[17/35] Lyon...
-> 3 hôtels | total cumulé : 51
[18/35] Gorges du Verdon...
-> 3 hôtels | total cumulé : 54
[19/35] Bormes les Mimosas...
-> 3 hôtels | total cumulé : 57
[20/35] Cassis...
-> 3 hôtels | total cumulé : 60
[21/35] Marseille...
-> 3 hôtels | total cumulé : 63
[22/35] Aix en Provence...
-> 3 hôtels | total cumulé : 66
[23/35] Avignon...
-> 3 hôtels | total cumulé : 69
[24/35] Uzes...
-> 3 hôtels | total cumulé : 72
[25/35] Nimes...
-> 3 hôtels | total cumulé : 75
[26/35] Aigues Mortes...
-> 3 hôtels | total cumulé : 78
[27/35] Saintes Maries de la Mer...
-> 3 hôtels | total cumulé : 81
[28/35] Collioure...
-> 3 hôtels | total cumulé : 84
[29/35] Carcassonne...
-> 3 hôtels | total cumulé : 87
[30/35] Ariege...
-> 3 hôtels | total cumulé : 90
[31/35] Toulouse...
-> 3 hôtels | total cumulé : 93
[32/35] Montauban...


```
-> 3 hôtels | total cumulé : 96
[33/35] Biarritz...
-> 3 hôtels | total cumulé : 99
[34/35] Bayonne...
-> 3 hôtels | total cumulé : 102
[35/35] La Rochelle...
-> 3 hôtels | total cumulé : 105
```

Scraping terminé - 105 lignes

	city	hotel_name	url	rating	description	city_id
0	Mont Saint Michel	Auberge Saint Pierre	https://www.booking.com/hotel/fr/auberge-saint...	8.0	N/A	1
1	Mont Saint Michel	Appart Standing - La Coque d'Or - Mont-St-Michel	https://www.booking.com/hotel/fr/la-coque-d-or...	9.6	N/A	1
2	Mont Saint Michel	Chez Adèle, au pied de l'Abbaye	https://www.booking.com/hotel/fr/chez-adele-le...	9.1	N/A	1
3	St Malo	Studio Les Armateurs 3 by Interhome	https://www.booking.com/hotel/fr/studio-les-ar...	8.0	N/A	2
4	St Malo	Appartement lumineux et calme proche mer a pieds	https://www.booking.com/hotel/fr/appartement-l...	9.7	N/A	2

5. Nettoyage, géocodage des hôtels, agrégation et score de voyage

Nettoyage : suppression des lignes sans note, deduplication par `(hotel_name, city)`. **Géocodage des hôtels** : le cahier des charges demande la latitude / longitude de chaque hôtel. On interroge Nominatim sur "nom hôtel, ville, France" ; en cas d'echec, on retombe sur les coordonnées du centre-ville (fallback). **Score de voyage** : chaque critère (température, note hôtelière, faible pluie) est normalisée entre 0 et 1, puis pondère. Cette méthode est plus rigoureuse que de mélanger des échelles différentes.

In [] :

```
# --- Nettoyage ---
df_clean = pd.read_csv(DATA / "hotels_booking_35_villes.csv").copy()
df_clean = (df_clean
            .dropna(subset=["rating"])          # uniquement les hôtels notés
            .drop_duplicates(subset=["hotel_name", "city"]) # suppression des doublons
            )
df_clean["description"] = df_clean["description"].fillna("N/A")

# --- Géocodage de chaque hôtel (coordonnées propres à l'établissement) ---
# Récupération des coordonnées de centre-ville pour le fallback
df_city_coords = df_weather_final[["city", "latitude", "longitude"]].rename(
    columns={"latitude": "city_lat", "longitude": "city_lon"})
df_clean = df_clean.merge(df_city_coords, on="city", how="left")

geo_hotels = Nominatim(user_agent="kayak_hotels_agent")
hotel_lat, hotel_lon = [], []

for _, h in df_clean.iterrows():
    try:
        loc = geo_hotels.geocode(f"{h['hotel_name']}, {h['city']}, France")
        if loc:
            hotel_lat.append(loc.latitude)
            hotel_lon.append(loc.longitude)
        else:
            # Fallback : coordonnées du centre-ville
            hotel_lat.append(h["city_lat"])
            hotel_lon.append(h["city_lon"])
    except Exception:
        hotel_lat.append(h["city_lat"])
        hotel_lon.append(h["city_lon"])
    time.sleep(1) # Nominatim : 1 requête/seconde

df_clean["latitude"] = hotel_lat
```

```

df_clean["longitude"] = hotel_lon
df_clean = df_clean.drop(columns=["city_lat", "city_lon"])
df_clean.to_csv(DATA / "hotels_cleaned_full.csv", index=False)

# --- Score agrégé par ville ---
df_city_scores = (df_clean
    .groupby(["city_id", "city"])["rating"]
    .mean().round(2)
    .reset_index()
    .rename(columns={"rating": "mean_hotel_rating"})
)
df_city_scores.to_csv(DATA / "villes_scores_aggregated.csv", index=False)

# --- Fusion météo + hôtels ---
# LEFT JOIN : on garde toutes les villes même sans hôtel scrapé
df_final = pd.merge(df_weather_final, df_city_scores, on=["city_id", "city"], how="left")

# --- Calcul du travel_score (normalisation min-max puis pondération) ---
def minmax(s):
    etendue = s.max() - s.min()
    return (s - s.min()) / etendue if etendue else s * 0

df_final["temp_norm"] = minmax(df_final["temp"]) # plus chaud = mieux
df_final["rating_norm"] = minmax(df_final["mean_hotel_rating"]) # mieux noté = mieux
df_final["rain_norm"] = 1 - minmax(df_final["rain_prob"]) # moins de pluie = mieux

# Pondération : 40% température, 40% qualité hôtelière, 20% faible pluie
df_final["travel_score"] = (
    0.4 * df_final["temp_norm"]
    + 0.4 * df_final["rating_norm"]
    + 0.2 * df_final["rain_norm"]
) * 100 # échelle 0-100 lisible

df_final = df_final.sort_values("travel_score", ascending=False)
df_final.to_csv(DATA / "kayak_final_enriched_data.csv", index=False)

print(f"Dataset final : {df_final.shape}")
display(df_final.head(10))

```

Dataset final : (35, 13)

	city_id	city	latitude	longitude	temp	rain_prob	rain_volume	weather_description	mean_hotel_rating	temp_norm	rating_norm	rain_norm	travel_score
21	22	Aix en Provence	43.529842	5.447474	29.6	0.01	0.1	ciel dégagé	8.40	0.961538	0.755668	0.975	88.188239
23	24	Uzes	44.012128	4.419672	28.6	0.05	2.1	ciel dégagé	8.80	0.884615	0.856423	0.875	87.141542
26	27	Saintes Maries de la Mer	43.451592	4.427720	28.1	0.00	0.0	ciel dégagé	8.60	0.846154	0.806045	1.000	86.087967
27	28	Collioure	42.525050	3.083155	28.0	0.06	1.6	ciel dégagé	8.40	0.838462	0.755668	0.850	80.765162
22	23	Avignon	43.949249	4.805901	29.7	0.03	1.4	ciel dégagé	7.43	0.969231	0.511335	0.925	77.722631
28	29	Carcassonne	43.213036	2.349107	25.2	0.07	7.6	nuageux	9.00	0.623077	0.906801	0.825	77.695117
20	21	Marseille	43.296399	5.377789	28.7	0.00	0.0	ciel dégagé	7.30	0.892308	0.478589	1.000	74.835885
31	32	Montauban	44.017584	1.354999	23.5	0.05	25.5	nuageux	8.73	0.492308	0.838791	0.875	70.743945
13	14	Dijon	47.321581	5.041470	23.9	0.18	8.0	légère pluie	9.23	0.523077	0.964736	0.550	70.512498
17	18	Gorges du Verdon	43.749656	6.328562	23.9	0.01	0.1	ciel dégagé	8.37	0.523077	0.748111	0.975	70.347510

6. Data Lake : AWS S3

Clés AWS chargées depuis `config/aws.env` . Les 4 fichiers CSV transformés sont envoyés dans le bucket S3 (couches silver / gold du Data Lake).

```
In [ ]: # Chargement des clés AWS depuis Le fichier .env (jamais en vraie dans Le code)
load_dotenv(dotenv_path=Path("config/aws.env"), override=True)

ACCESS_KEY = os.getenv("AWS_ACCESS_KEY_ID")
SECRET_KEY = os.getenv("AWS_SECRET_ACCESS_KEY")
BUCKET_NAME = os.getenv("S3_BUCKET_NAME")
REGION     = os.getenv("AWS_REGION", "eu-west-3")

# Session AWS authentifiée
session = boto3.Session(aws_access_key_id=ACCESS_KEY,
                        aws_secret_access_key=SECRET_KEY,
                        region_name=REGION)
s3      = session.resource("s3")
s3_client = session.client("s3")

# Création du bucket S3 s'il n'existe pas encore (role de Data Lake : stockage brut scalable)
try:
    s3_client.head_bucket(Bucket=BUCKET_NAME)
    print(f"Bucket '{BUCKET_NAME}' déjà existant.")
except:
    s3.create_bucket(Bucket=BUCKET_NAME,
                    CreateBucketConfiguration={"LocationConstraint": REGION})
    print(f"Bucket '{BUCKET_NAME}' créé.")

# Upload des 4 fichiers CSV transformés vers S3
for fname in [
    "villes_meteo_coords.csv",
    "hotels_cleaned_full.csv",
    "villes_scores_aggregated.csv",
    "kayak_final_enriched_data.csv"
]:
    local_path = DATA / fname
    if local_path.exists():
        s3.Bucket(BUCKET_NAME).upload_file(str(local_path), fname) # clé S3 = nom du fichier
        print(f"{fname} envoyé")
    else:
        print(f"{fname} non trouvé localement")
```

7. Data Warehouse : AWS RDS (PostgreSQL)

Clés chargées depuis `config/aws.env` . Deux tables relationnelles : `cities` (1 ligne / ville) et `hotels` (N lignes / ville, liées par `city_id`).

```
In [ ]: # Chargement des identifiants RDS depuis Le fichier .env
load_dotenv(dotenv_path=Path("config/aws.env"), override=True)

DB_USER = os.getenv("RDS_USERNAME")
DB_PASSWORD = os.getenv("RDS_PASSWORD")
DB_HOST = os.getenv("RDS_HOSTNAME") # Endpoint de l'instance RDS
DB_PORT = os.getenv("RDS_PORT", "5432") # Port par défaut de PostgreSQL
DB_NAME = os.getenv("RDS_DB_NAME", "postgres") # Nom de la base de données

# Connexion via SQLAlchemy : postgresql://user:password@host:port/database
engine = create_engine(f"postgresql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}")

try:
    # Table cities : 1 ligne par ville (coordonnées, météo, score)
    df_final.to_sql("cities", con=engine, if_exists="replace", index=False)
    print(f"Table 'cities' injectée ({len(df_final)} destinations)")
```

```
# Table hôtels : N lignes par ville, Liées par city_id (relation one-to-many)
df_clean.to_sql("hotels", con=engine, if_exists="replace", index=False)
print(f"Table 'hotels' injectée ({len(df_clean)} établissements)")

except Exception as e:
    print(f"Erreur RDS : {e}")
    print("-> Vérifier le Security Group AWS (port 5432, IP autorisée)")
```

8. Visualisation : Cartes interactives Plotly

Carte 1 : Top 5 destinations (score météo + hôtels)

```
In [8]: # --- Tableau Top 5 destinations ---
top_5_display = df_final.head(5)[["city", "temp", "rain_prob", "mean_hotel_rating", "travel_score"]].copy()
top_5_display.columns = ["Destination", "Température (C)", "Prob. pluie", "Note hôtels", "Score voyage"]
top_5_display = top_5_display.round(2).reset_index(drop=True)
top_5_display.index += 1
display(top_5_display)

# --- Carte interactive : Top 5 destinations ---
top_5 = df_final.head(5)
fig = px.scatter_mapbox(
    top_5,
    lat="latitude", lon="longitude",
    hover_name="city",
    hover_data={
        "latitude": False, "longitude": False,
        "temp": ":.1f",
        "rain_prob": ":.2f",
        "mean_hotel_rating": ":.1f"
    },
    color="temp",
    size="travel_score",
    color_continuous_scale=px.colors.sequential.YlOrRd,
    size_max=18, zoom=5,
    mapbox_style="open-street-map",
    title="Top 5 Destinations - Meilleur compromis Météo / Hôtels"
)
fig.update_layout(margin={"r": 10, "t": 50, "l": 10, "b": 10})
fig.show()
```

	Destination	Température (C)	Prob. pluie	Note hôtels	Score voyage
1	Aix en Provence	29.6	0.01	8.40	88.19
2	Uzes	28.6	0.05	8.80	87.14
3	Saintes Maries de la Mer	28.1	0.00	8.60	86.09
4	Collioure	28.0	0.06	8.40	80.77
5	Avignon	29.7	0.03	7.43	77.72

Carte 2 : Top 20 hôtels les mieux notés

```
In [ ]: # --- Tableau Top 20 hôtels ---
top_20 = df_clean.sort_values("rating", ascending=False).head(20)

top_20_display = top_20[["hotel_name", "city", "rating", "url1"]].copy()
```

```
top_20_display.columns = ["Hôtel", "Ville", "Note", "URL"]
top_20_display = top_20_display.reset_index(drop=True)
top_20_display.index += 1
display(top_20_display)

# --- Carte interactive : Top 20 hôtels (coordonnées propres à chaque hôtel) ---
fig2 = px.scatter_mapbox(
    top_20,
    lat="latitude", lon="longitude",
    hover_name="hotel_name",
    hover_data={
        "latitude": False, "longitude": False,
        "city": True,
        "rating": ":.1f"
    },
    color="rating",
    size="rating",
    color_continuous_scale=px.colors.sequential.Viridis,
    zoom=5, mapbox_style="open-street-map",
    title="<b>Top 20 Hôtels les mieux notés en France</b>"
)
fig2.update_layout(margin={"r": 10, "t": 50, "l": 10, "b": 10})
fig2.show()
```

	Hôtel	Ville	Note	URL
1	Gîtes Strengbach	Chateau du Haut Koenigsbourg	9.7	https://www.booking.com/hotel/fr/strenbach.fr....
2	Villa haut de gamme Montauban	Montauban	9.7	https://www.booking.com/hotel/fr/villa-haut-de...
3	Appartement lumineux et calme proche mer a pieds	St Malo	9.7	https://www.booking.com/hotel/fr/appartement-l...
4	Le petit POTE' studio Dijon hyper centre	Dijon	9.6	https://www.booking.com/hotel/fr/le-petit-pote...
5	La Fermette vosgienne de la Goutte	Chateau du Haut Koenigsbourg	9.6	https://www.booking.com/hotel/fr/fermette-vosg...
6	Le petit Rouen	Rouen	9.6	https://www.booking.com/hotel/fr/le-petit-roue...
7	Appartement PETIT SAINT GIMER	Carcassonne	9.6	https://www.booking.com/hotel/fr/appartement-p...
8	Appart Standing - La Coque d'Or - Mont-St-Michel	Mont Saint Michel	9.6	https://www.booking.com/hotel/fr/la-coque-d-or...
9	Maison cosy au centre historique de Bayeux	Bayeux	9.5	https://www.booking.com/hotel/fr/maison-cosy-a...
10	"La Suite" 3 Etoiles	Uzes	9.5	https://www.booking.com/hotel/fr/la-suite-a-50...
11	Gîtes Au Château Fleuri	Eguisheim	9.5	https://www.booking.com/hotel/fr/gite-au-chate...
12	Le Studio des Vignes	Eguisheim	9.5	https://www.booking.com/hotel/fr/charmant-stud...
13	Appartement 4 personnes dans domaine privé, ca...	Bormes les Mimosas	9.5	https://www.booking.com/hotel/fr/t2-et-studio-...
14	Charmante maison de pêcheur	Aigues Mortes	9.4	https://www.booking.com/hotel/fr/charmante-mai...
15	Résidence Brocéliande	St Malo	9.4	https://www.booking.com/hotel/fr/residence-bro...
16	Studio Centre Ville Office De Tourisme	Bayonne	9.4	https://www.booking.com/hotel/fr/studio-centre...
17	Chambre d'Hôtes La Villa Molina	Besancon	9.4	https://www.booking.com/hotel/fr/la-villa-moli...
18	L'océan à perte de vue, la grande plage à vos ...	Biarritz	9.4	https://www.booking.com/hotel/fr/l-39-ocean-a-...
19	Annecy centre et lac	Annecy	9.3	https://www.booking.com/hotel/fr/annecy-centre...
20	Cosy T2 38m2 - Centre-Ville Dijon- Gare/Darcy	Dijon	9.3	https://www.booking.com/hotel/fr/9-rue-benigne...

9. Conformité au RGPD

Le processus de collecte a été conçu dans le respect du Règlement Général sur la Protection des Données.

- **Absence de données personnelles.** Le projet ne collecte aucune donnée relative à une personne physique identifiée ou identifiable. Les données traitées sont des informations publiques et non nominatives : coordonnées géographiques de villes, prévisions météorologiques, et fiches d'établissements hôteliers (nom commercial, note agrégée, description, lien). Les avis individuels et les noms d'utilisateurs ne sont ni collectés ni stockés.
- **Minimisation des données.** Seuls les champs strictement nécessaires à la recommandation sont conservés (ville, coordonnées, température, probabilité de pluie, note moyenne des hôtels). Aucune donnée superflue n'est extraite.
- **Sécurité des secrets.** Les clés d'API (OpenWeatherMap) et les identifiants AWS (S3, RDS) ne figurent jamais en clair dans le code. Ils sont chargés depuis des fichiers `.env` exclus du versionnement par le fichier `.gitignore`. Seuls des fichiers d'exemple (`openweather.example`, `aws.env.example`) sont publiés.
- **Sources et conditions d'utilisation.** Les données météo proviennent d'une API publique utilisée dans le cadre de ses conditions. Le scraping de Booking.com se limite à des informations publiquement affichées, avec des cadences d'appel raisonnables (pauses entre requêtes) afin de ne pas surcharger le service.
- **Stockage maîtrisé.** Les données sont hébergées sur une infrastructure AWS dont l'accès est restreint (identifiants dédiés, groupe de sécurité RDS limitant les adresses IP autorisées).

10. Conclusion

Ce projet met en oeuvre un pipeline ETL complet :

Etape	Technologie
Géocodage des villes et des hôtels	Nominatim (OpenStreetMap)
Données météo (agrégées sur l'horizon de prevision)	API OpenWeatherMap
Scraping hôtels	Selenium + BeautifulSoup
Stockage brut	AWS S3 (Data Lake)
Stockage structure	AWS RDS / PostgreSQL (Data Warehouse)
Visualisation	Plotly Express (Mapbox)

Pistes d'amélioration : automatisation hebdomadaire via AWS Lambda + enrichissement avec les prix des billets de train (API SNCF) pour un score de voyage plus complet.